CyclicLR#

https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CyclicLR.

Description:

Sets the learning rate of each parameter group according to cyclical learning rate policy (CLR). The policy cycles the learning rate between two boundaries with a constant frequency, as detailed in the paper Cyclical Learning Rates for Training Neural Networks. The distance between the two boundaries can be scaled on a per-iteration or per-cycle basis. Cyclical learning rate policy changes the learning rate after every batch. step should be called after a batch has been used for training. This class has three built-in policies, as put forth in the paper: “triangular”: A basic triangular cycle without amplitude scaling.

Function Signature

```
class torch.optim.lr_scheduler.CyclicLR(optimizer, base_lr,
max_lr, step_size_up=2000, step_size_down=None,
mode='triangular', gamma=1.0, scale_fn=None,
scale_mode='cycle', cycle_momentum=True, base_momentum=0.8,
max_momentum=0.9, last_epoch=-1)[source]#

Parameters

get_last_lr()[source]#

Return type

get_lr()[source]#
```

Returns:

dict[str, Any]

Code Examples

```
>>> optimizer = torch.optim.SGD(model.parameters(), lr=0.1,
momentum=0.9)
>>> scheduler = torch.optim.lr_scheduler.CyclicLR(
...     optimizer,
...     base_lr=0.01,
...     max_lr=0.1,
...     step_size_up=10,
... )
>>> data_loader = torch.utils.data.DataLoader(...)
>>> for epoch in range(10):
>>>     for batch in data_loader:
>>>         train_batch(...)
>>>         scheduler.step()
```

Detailed Documentation

Documentation

Sets the learning rate of each parameter group according to cyclical learning rate policy (CLR).

The policy cycles the learning rate between two boundaries with a constant frequency, as detailed in the paper Cyclical Learning Rates for Training Neural Networks. The distance between the two boundaries can be scaled on a per-iteration or per-cycle basis.

Cyclical learning rate policy changes the learning rate after every batch. step should be

called after a batch has been used for training.

This class has three built-in policies, as put forth in the paper:

“triangular”: A basic triangular cycle without amplitude scaling.

“triangular2”: A basic triangular cycle that scales initial amplitude by half each cycle.

“exp_range”: A cycle that scales initial amplitude by gammacycle iterations\text{gamma}^{\text{cycle iterations}}gammacycle iterations at each cycle iteration.

This implementation was adapted from the github repo: bckenstler/CLR

optimizer (Optimizer) – Wrapped optimizer.

base_lr (float or list) – Initial learning rate which is the lower boundary in the cycle for each parameter group.

max_lr (float or list) – Upper learning rate boundaries in the cycle for each parameter group. Functionally, it defines the cycle amplitude (max_lr - base_lr). The lr at any cycle is the sum of base_lr and some scaling of the amplitude; therefore max_lr may not actually be reached depending on scaling function.

step_size_up (int) – Number of training iterations in the increasing half of a cycle. Default: 2000

step_size_down (int) – Number of training iterations in the decreasing half of a cycle. If step_size_down is None, it is set to step_size_up. Default: None

mode (str) – One of {triangular, triangular2, exp_range}. Values correspond to policies detailed above. If scale_fn is not None, this argument is ignored. Default: ‘triangular’

gamma (float) – Constant in ‘exp_range’ scaling function: gamma**(cycle iterations) Default: 1.0

scale_fn (function) – Custom scaling policy defined by a single argument lambda function, where 0 <= scale_fn(x) <= 1 for all x >= 0. If specified, then ‘mode’ is ignored. Default: None

scale_mode (str) – {‘cycle’, ‘iterations’}. Defines whether scale_fn is evaluated on cycle number or cycle iterations (training iterations since start of cycle). Default: ‘cycle’

cycle_momentum (bool) – If True, momentum is cycled inversely to learning rate between ‘base_momentum’ and ‘max_momentum’. Default: True

base_momentum (float or list) – Lower momentum boundaries in the cycle for each parameter group. Note that momentum is cycled inversely to learning rate; at the peak of a cycle, momentum is ‘base_momentum’ and learning rate is ‘max_lr’. Default: 0.8

max_momentum (float or list) – Upper momentum boundaries in the cycle for each parameter group. Functionally, it defines the cycle amplitude (max_momentum - base_momentum). The momentum at any cycle is the difference of max_momentum and some scaling of the amplitude; therefore base_momentum may not actually be reached depending on scaling function. Note that momentum is cycled inversely to learning rate; at the start of a cycle, momentum is ‘max_momentum’ and learning rate is ‘base_lr’ Default: 0.9

last_epoch (int) – The index of the last batch. This parameter is used when resuming a training job. Since step() should be invoked after each batch instead of after each epoch, this number represents the total number of batches computed, not the total number of epochs computed. When last_epoch=-1, the schedule is started from the beginning. Default: -1

Return last computed learning rate by current scheduler.

Calculate the learning rate at batch index.

This function treats self.last_epoch as the last batch index.

If self.cycle_momentum is True, this function has a side effect of updating the optimizer's momentum.

Load the scheduler's state.

Get the scaling policy.

Return the state of the scheduler as a dict.

It contains an entry for every variable in self.__dict__ which is not the optimizer. The learning rate lambda functions will only be saved if they are callable objects and not if they are functions or lambdas.

When saving or loading the scheduler, please make sure to also save or load the state of the optimizer.